

LAPORAN PRAKTIKUM
PEMROGRAMAN BERBASIS OBJEK
Relasi Antar Objek (Class Diagram & Asosiasi)

Disusun Oleh:

Nofri Ilham

2511531013

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

Ezza Andini



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum yang berjudul “Relasi Antar Objek (Class Diagram & Asosiasi)” dapat diselesaikan dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pelaksanaan tugas praktikum pada mata kuliah terkait pemodelan atau perancangan perangkat lunak berorientasi objek. Melalui praktikum ini, penulis berusaha memahami bagaimana objek-objek dalam sebuah sistem saling berhubungan dan bagaimana hubungan tersebut direpresentasikan dalam bentuk class diagram.

Dalam praktikum ini, penulis mempelajari konsep relasi antar objek, khususnya asosiasi. Asosiasi digunakan untuk menggambarkan hubungan antara dua class, misalnya hubungan antara class Mahasiswa dan MataKuliah, di mana seorang mahasiswa dapat mengambil lebih dari satu mata kuliah dan satu mata kuliah dapat diikuti oleh banyak mahasiswa. Hubungan semacam ini sering disertai dengan multiplicity untuk menunjukkan jumlah objek yang saling berhubungan.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi pemahaman materi, proses pembuatan class diagram, maupun cara penulisan laporan. Oleh karena itu, kritik dan saran yang membangun dari dosen pengampu dan asisten praktikum sangat penulis harapkan demi perbaikan di kemudian hari.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta teman-teman yang telah membantu selama proses praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat bagi penulis pribadi maupun bagi pembaca yang ingin memahami relasi antar objek dalam class diagram.

Padang, 22 September 2026

1.2 Tujuan

1. Agar dapat memahami konsep relasi antar objek dalam pemodelan berorientasi objek.
2. Agar mahasiswa mampu memahami fungsi class diagram dalam menggambarkan class, atribut, operasi, serta hubungan antar objek dalam sebuah sistem.
3. Agar mahasiswa dapat memahami pengertian dan fungsi asosiasi sebagai hubungan antar class.
4. Agar mahasiswa mampu membuat class diagram yang menunjukkan asosiasi antar class beserta multiplicity-nya.
5. Agar mahasiswa dapat menerapkan pemahaman relasi antar objek dalam perancangan sistem atau program berorientasi objek.

1.3 Manfaat

1. Menambah pemahaman penulis mengenai relasi antar objek, khususnya hubungan asosiasi antar class.
2. Melatih kemampuan penulis dalam menganalisis sebuah studi kasus dan mengubahnya menjadi class diagram.
3. Meningkatkan keterampilan penulis dalam membuat class diagram yang terstruktur, mencakup class, atribut, operasi, dan asosiasi.
4. Membantu penulis memahami penggunaan multiplicity untuk menunjukkan jumlah hubungan antar objek, misalnya satu mahasiswa dapat mengambil banyak mata kuliah.

5. Menjadi dasar pengetahuan bagi penulis dalam merancang program berorientasi objek yang memiliki hubungan antar objek.

BAB II

PEMBAHASAN

2.1 Tahapan Pelaksanaan Praktikum

2.1.1 Membuat Class Transaksi

Pada tahap pertama, dibuat class Transaksi yang digunakan untuk menyimpan informasi mengenai transaksi yang dilakukan pada sebuah rekening. Class Transaksi memiliki tiga atribut, yaitu idTransaksi, jenis, dan nominal.

Atribut idTransaksi digunakan untuk menyimpan identitas dari setiap transaksi, atribut jenis digunakan untuk menentukan jenis transaksi, sedangkan atribut nominal digunakan untuk menyimpan jumlah uang yang ditransaksikan. Jenis transaksi yang digunakan terdiri dari Debit untuk penarikan dan Kredit untuk penyetoran.

Selanjutnya, dibuat constructor Transaksi untuk menginisialisasi nilai dari ketiga atribut tersebut. Selain itu, dibuat method cetakDetail() yang digunakan untuk menampilkan informasi transaksi.

```

package Pertemuan2;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    //Konstruktor
    public Transaksi(String id,String Jenis, double nominal )
    {
        this.idTransaksi = id;
        this.jenis=Jenis;
        this.nominal =nominal;
    }
    public void cetakDetail() {
        System.out.println("ID: "+ idTransaksi +" | Jenis: "+ jenis + " | Nominal: Rp"+ nominal);
    }
}

```

2.1.2 Membuat Relasi pada Class Rekening

Pada tahap kedua, dilakukan penambahan relasi antara class Rekening dengan class Transaksi. Relasi yang digunakan adalah asosiasi one-to-many, yaitu satu rekening dapat memiliki banyak transaksi.

Untuk menerapkan relasi tersebut, ditambahkan atribut riwayatTransaksi dengan tipe `ArrayList<Transaksi>` pada class Rekening.

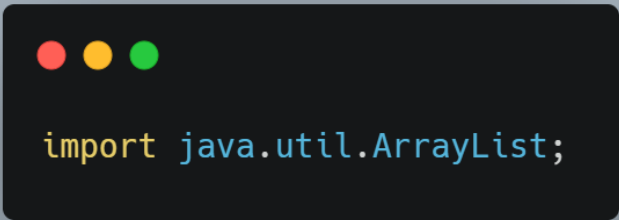
```

//Implementasi Asosiasi (1-to-many)
    ArrayList<Transaksi>riwayatTransaksi;

```

Penggunaan `ArrayList` memungkinkan satu objek Rekening menyimpan beberapa objek Transaksi di dalamnya. Hal ini sesuai dengan konsep hubungan has-a, yaitu sebuah rekening memiliki riwayat transaksi.

Agar `ArrayList` dapat digunakan, ditambahkan import:



```
import java.util.ArrayList;
```

Kemudian pada constructor Rekening, riwayatTransaksi diinisialisasi menggunakan:



```
//Wajib meninisialisasi ArrayList di dalam konstruktor agar tidak NullPointerException  
this.riwayatTransaksi=new ArrayList<>();
```

Inisialisasi tersebut diperlukan agar riwayatTransaksi dapat digunakan untuk menyimpan objek Transaksi.

2.1.3 Mencatat Riwayat Transaksi Setor Tunai

Pada tahap ketiga, dilakukan pencatatan riwayat transaksi ketika pengguna melakukan setor tunai. Pada praktikum ini, setiap kali terjadi transaksi penyetoran, program akan membuat objek Transaksi baru dan menyimpannya ke dalam riwayatTransaksi. Hal tersebut merupakan bagian dari tujuan praktikum untuk membuat objek Transaksi setiap kali terjadi aktivitas penyetoran atau penarikan dan menyimpannya di dalam objek Rekening.

Pada method setorTunai(), terlebih dahulu dilakukan pengecekan agar nominal setor lebih dari 0. Jika kondisi terpenuhi, saldo rekening akan ditambahkan dengan nominal setoran.

Selanjutnya, dibuat ID transaksi dengan prefiks TRX-S-, kemudian dibuat objek Transaksi dengan jenis Kredit.



```
//Merekam riwayat (Pembuatan objek Transaksi di dalam method)
String idTrx="TRX-S-"+ System.currentTimeMillis();
Transaksi trxBaru =new Transaksi(idTrx,"Kredit",nominal);
riwayatTransaksi.add(trxBaru);
```

Objek Transaksi yang telah dibuat kemudian dimasukkan ke dalam ArrayList riwayatTransaksi menggunakan method add().

Dengan demikian, setiap transaksi setor tunai yang berhasil tidak hanya mengubah saldo rekening, tetapi juga menghasilkan objek transaksi yang disimpan sebagai riwayat pada objek Rekening.

2.2 Latihan/Tugas

2.2.1 Tugas 1. Integrasi Rekam Jejak Penarikan

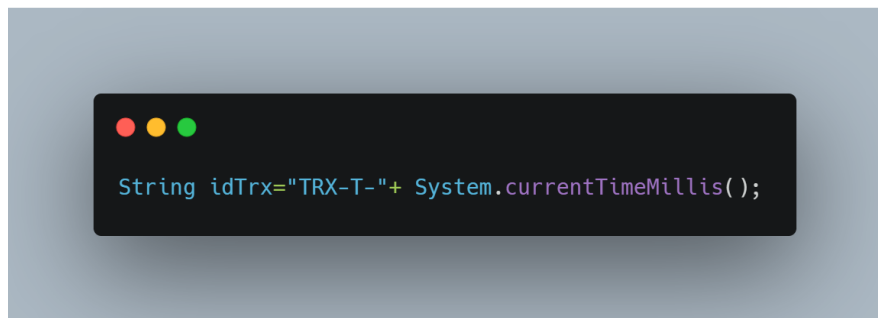
Tugas Pertama, dilakukan pengembangan pada method tarikTunai(double nominal) yang terdapat pada class Rekening. Pengembangan ini bertujuan agar setiap transaksi penarikan tunai yang berhasil dapat disimpan ke dalam riwayat transaksi rekening. Transaksi penarikan harus dicatat sebagai jenis Debit dan menggunakan ID transaksi dengan prefiks TRX-T-. Pencatatan hanya dilakukan apabila penarikan berhasil dan saldo mencukupi.

Pada program, terlebih dahulu dilakukan pengecekan terhadap nominal penarikan. Jika nominal kurang dari Rp10.000, maka transaksi dinyatakan gagal. Program juga melakukan pengecekan apakah nominal penarikan lebih besar daripada saldo yang tersedia. Jika saldo tidak mencukupi, transaksi tidak dapat dilakukan.

Apabila seluruh kondisi terpenuhi, saldo rekening akan dikurangi sesuai nominal penarikan menggunakan perintah:



Setelah saldo berhasil dikurangi, program membuat ID transaksi secara otomatis menggunakan `System.currentTimeMillis()` dengan prefiks TRX-T-.



Selanjutnya dibuat objek Transaksi baru dengan jenis transaksi Debit:



Objek transaksi tersebut kemudian dimasukkan ke dalam `riwayatTransaksi` menggunakan method `add()`:



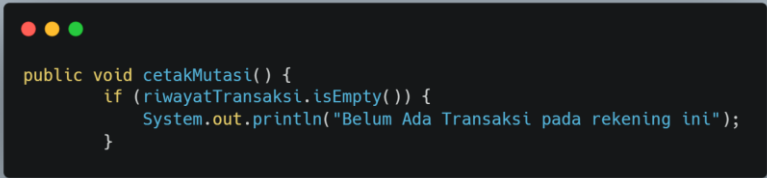
```
riwayatTransaksi.add(trxBaru);
```

Transaksi penarikan yang berhasil tidak hanya mengurangi saldo rekening, tetapi juga tercatat sebagai riwayat transaksi. Jika penarikan gagal karena nominal tidak memenuhi ketentuan atau saldo tidak mencukupi, objek Transaksi tidak dibuat dan tidak dimasukkan ke dalam riwayat.

2.2.2 Tugas 2. Membuat Fitur Cetak Mutasi Rekening

Kedua, dibuat fitur cetak mutasi rekening yang digunakan untuk menampilkan seluruh riwayat transaksi yang telah dilakukan oleh rekening. Sesuai instruksi modul, method yang dibuat pada class Rekening adalah cetakMutasi(). Method tersebut menggunakan perulangan for-each untuk menelusuri seluruh objek Transaksi yang tersimpan di dalam riwayatTransaksi.

Pada program, terlebih dahulu dilakukan pengecekan apakah riwayatTransaksi kosong menggunakan method isEmpty().



```
public void cetakMutasi() {  
    if (riwayatTransaksi.isEmpty()) {  
        System.out.println("Belum Ada Transaksi pada rekening ini");  
    }  
}
```

Jika belum terdapat transaksi, program akan menampilkan pesan bahwa belum ada transaksi pada rekening tersebut.

Apabila terdapat transaksi, program menggunakan perulangan for-each untuk mengambil setiap objek Transaksi yang terdapat dalam riwayatTransaksi.



Setiap objek Transaksi kemudian menjalankan method cetakDetail() untuk menampilkan informasi berupa ID transaksi, jenis transaksi, dan nominal transaksi.

Setelah method cetakMutasi() dibuat, fitur tersebut diintegrasikan ke dalam class Main melalui penambahan menu 6. Cetak Mutasi. Ketika pengguna memilih menu tersebut, program akan memeriksa apakah terdapat akunAktif. Jika akun aktif tersedia, method cetakMutasi() akan dipanggil melalui:



2.3 Output

```
=== SISTEM PERBANKAN MINI===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi
7. Cek Transaksi
0. Keluar
Pilih Menu:
1
Masukkan No Rekening:
1234
Masukkan Nama Pemilik:
nofri
Masukkan Saldo Awal:
100000
    Rekening atas nama nofri Berhasil dibuat dengan saldo Rp 100000.0
Rekening berhasil ditambahkan.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi
7. Cek Transaksi
0. Keluar
Pilih Menu:
2
Masukkan nominal setor:
100000
    Setor tunai Rp 100000.0 Berhasil. Saldo saat ini: Rp 200000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi
7. Cek Transaksi
0. Keluar
Pilih Menu:
3
```

```
Pilih Menu:
3
Masukkan Nominal Tarik tunai
50000
Transaksi Berhasil: Saldo Anda: Rp 150000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi
7. Cek Transaksi
0. Keluar
Pilih Menu:
6
ID: TRX-S-1790076036604 | Jenis: Kredit | Nominal: Rp100000.0
ID: TRX-T-1790076045486 | Jenis: Debit | Nominal: Rp50000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi
7. Cek Transaksi
0. Keluar
Pilih Menu:
```

2.4 Link Repository

https://github.com/nofriilham/PrakPBO_C_2511531013/tree/Pertemuan2

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan dengan judul “Relasi Antar Objek (Class Diagram & Asosiasi)”, dapat disimpulkan bahwa relasi antar objek dapat diterapkan dalam pemrograman berorientasi objek dengan menghubungkan class Rekening dan class Transaksi menggunakan konsep asosiasi one-to-many. Dalam penerapannya, satu objek Rekening dapat memiliki banyak objek Transaksi yang disimpan menggunakan `ArrayList<Transaksi>`.

Melalui praktikum ini, penulis juga dapat memahami penerapan class Transaksi yang memiliki atribut `idTransaksi`, `jenis`, dan `nominal`, serta method `cetakDetail()` untuk menampilkan informasi transaksi. Selain itu, proses setor dan tarik tunai dapat dikembangkan agar transaksi yang berhasil dapat dicatat ke dalam riwayat transaksi rekening.

Penerapan fitur rekam jejak penarikan dan cetak mutasi rekening juga menunjukkan bahwa objek transaksi dapat dibuat dan dikelola di dalam objek rekening. Dengan adanya fitur tersebut, transaksi setor maupun tarik dapat disimpan dan ditampilkan kembali melalui menu cetak mutasi. Praktikum ini memberikan pemahaman kepada penulis mengenai penerapan relasi antar objek dan penggunaan `ArrayList` dalam pengembangan program berorientasi objek.